5. TOPOLOGÍA DE RED

Para la creación del clúster[12] de servidores con Apache Spark, se han utilizado cinco máquinas virtuales. Una de las cinco máquinas será el servidor principal (Spark-Master) y el resto servidores esclavos (Spark-WorkerN). Las máquinas utilizadas son:

Hostname	IP (10.0.0.0/24)	Función
master	10.0.0.1	Servidor principal
worker1	10.0.0.2	Servidor esclavo
worker2	10.0.0.3	Servidor esclavo
worker3	10.0.0.4	Servidor esclavo

Para configurar la red de la máquinas editamos el fichero "*/etc/network/interfaces*":

```
# Configuración para la red interna
auto eth0
iface eth0 inet static
        address 10.0.0.X
        netmask 255.255.255.0
```

12 Un *clúster* consiste en la unión de varios servidor. los cuales trabajarán de forma paralela como si de un solo servidor se tratase.

6. CARACTERÍSTICAS Y REQUISITOS

Las máquinas virtuales utilizadas tendrán Debian 8.6 (Jessie) como sistema operativo y estarán virtualizadas con VirtualBox. Las principales características de Hardware son:

---------- MASTER ----------

- **Nº de cores**: 2
- **Memoria RAM**: 2048 Mb
- **Discos duros**: Disco duro virtual reservado dinámicamente
- **Tarjeta de red**: Dos adaptadores de red (Un adaptador puente a wlan0 y un adaptador para la red interna).

---------- WORKERS ----------

- **Nº de cores**: 1
- **Memoria RAM**: 1024 Mb
- **Discos duros**: Disco duro virtual reservado dinámicamente
- **Tarjeta de red**: Dos adaptadores de red (Un adaptador puente a wlan0 y un adaptador para la red interna).

Para el funcionamiento correcto del clúster de servidores, Apache Spark necesita la siguiente lista de software instalada:

- **Java**: vamos a instalar la versión 8.
- **Scala**: vamos a instalar la versión 2.11.8
- **OpenSSH**: Para permitir el acceso remoto por SSH entre las máquinas.
- **Apache Spark**: Spark pre-compilado para Hadoop.

7. INSTALACIÓN Y CONFIGURACIÓN DE SPARK EN DEBIAN

Vamos a instalar una serie de herramientas necesarias en todos los servidores a no ser que se indique lo contrario.

7.1 Instalación de Java

Añadimos los repositorios correspondientes:

```
root@master:/# echo "deb http://ppa.launchpad.net/webupd8team/java/ubuntu trusty main" > /etc/apt/sources.list.d/webupd8team-java.list
root@master:/# echo "deb-src http://ppa.launchpad.net/webupd8team/java/ubuntu trusty main" >> /etc/apt/sources.list.d/webupd8team-java.list
```

Importamos la siguiente clave para poder instalar JAVA 8:

```
root@master:/# apt-key adv --keyserver keyserver.ubuntu.com --recv-keys EEA14886
```

Actualizamos los repositorios:

```
root@master:/# apt-get update
```

Posteriormente instalamos JAVA 8 gracias a la herramienta *apt-get*:

```
root@master:/# apt-get install oracle-java8-installer
```

Comprobamos la versión de nuestro JAVA:

```
root@master:/# java -version

java version "1.8.0_121"
Java(TM) SE Runtime Environment (build 1.8.0_121-b13)
Java HotSpot(TM) 64-Bit Server VM (build 25.121-b13, mixed mode)
```

7.2 Instalación de Scala

Descargamos la última versión del paquete de instalación de scala, en nuestro caso *scala-2.13.0-M1.deb*:

```
root@master:/# wget https://downloads.lightbend.com/scala/2.13.0-M1/scala-2.13.0-M1.deb
```

Posteriormente, con la herramienta dpkg, instalamos el archivo descargado anteriormente:

```
root@master:/# dpkg -i scala-2.13.0-M1.deb
```

Comprobamos que se ha instalado correctamente scala y que se ejecuta su shell interactiva correctamente:

```
root@master:/# scala -version

Scala code runner version 2.13.0-M1 -- Copyright 2002-2017, LAMP/EPFL and Lightbend, Inc.

root@master:/# scala
```

```
root@master:/home/usuario# scala
Welcome to Scala 2.13.0-M1 (Java HotSpot(TM) 64-Bit Server VM, Java 1.8.0_121).
Type in expressions for evaluation. Or try :help.

scala> object HelloWorld {
     |     def main(args: Array[String]) {
     |       println("Hello, world!")
     |     }
     |   }
defined object HelloWorld

scala> HelloWorld.main(null)
Hello, world!

scala> :q
```

7.3 Instalación y configuración de OpenSSH

Apache Spark necesita acceso remoto sin pedir contraseña, para ello vamos a hacer uso de la herramienta SSH. En la máquina *master* instalamos el paquete openssh-server:

```
root@master:/# apt-get install openssh-server
```

En las máquinas workers permitimos el acceso remoto al usuario root:

```
root@master:/# nano /etc/ssh/sshd_config
PermitRootLogin yes
```

Posteriormente reiniciamos el servicio SSH:

```
root@master:/# service ssh restart
```

7.4 Instalación y configuración de Apache Spark

Descargamos e instalamos la última versión de Apache Spark de ESTE enlace:

```
root@master:/# wget http://d3kbcqa49mib13.cloudfront.net/spark-2.1.0-bin-hadoop2.7.tgz
```

Creamos el directorio */opt/spark*:

```
root@master:/# mkdir /opt/spark
```

Descomprimimos el fichero descargado anteriormente:

```
root@master:/# tar -xzvf spark-2.1.0-bin-hadoop2.7.tgz
```

Copiamos los ficheros de la carpeta descomprimida anteriormente a la nueva ubicación:

```
root@master:/# mv spark-2.1.0-bin-hadoop2.7/* /opt/spark/
```

Añadimos una nueva entrada en el fichero .bashrc:

```
root@master:/# nano ~/.bashrc

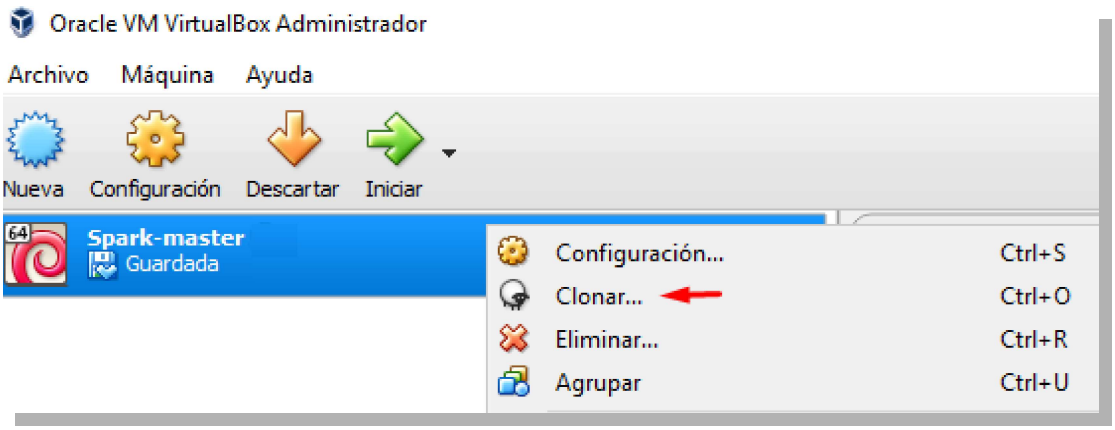
export SPARK_HOME=/opt/spark/
export PATH="/opt/spark/bin/:/opt/spark/sbin/:$PATH"
```

Cargamos las variables definidas anteriormente en el fichero *.bashrc*:

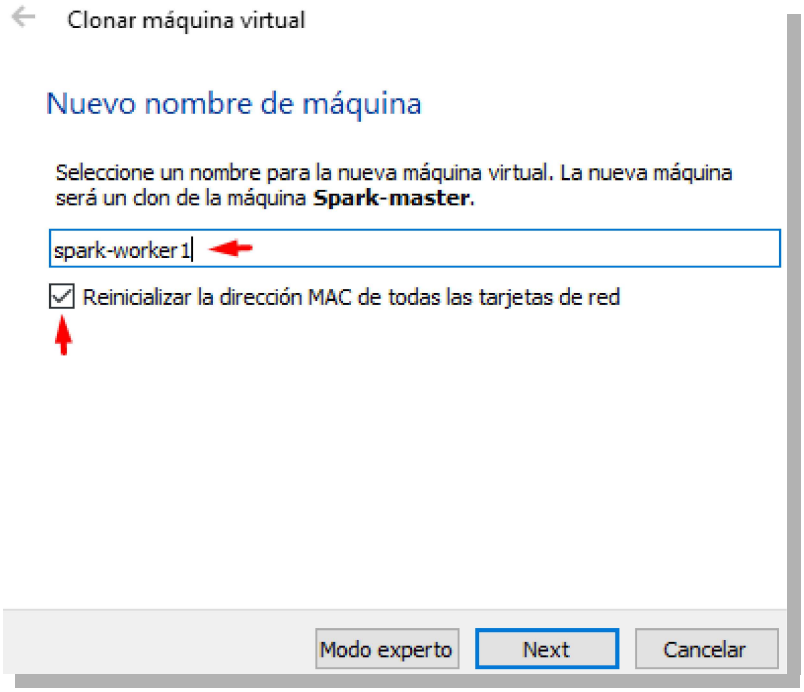
```
root@master:/# source ~/.bashrc
```

7.5. Creación de los servidores esclavos

Para no tener que realizar todas las acciones anteriores, se va a clonar el servidor master. Para ello, damos click derecho en la máquina *Spark-master* y pinchamos en el apartado *Clonar…:*

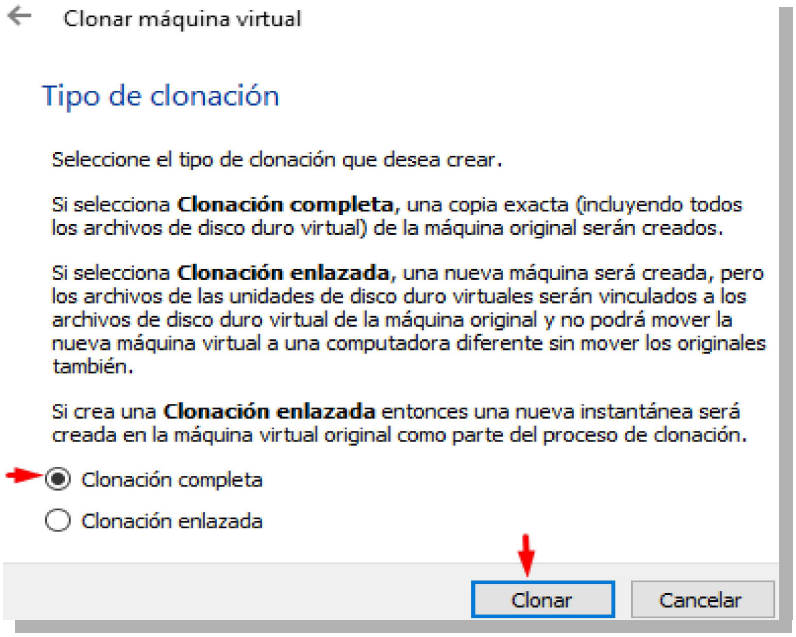


Posteriormente le asignamos un nombre a la máquina, en nuestro caso *spark-worker1*.



Muy Importante !!! : Recuerda reiniciar la MAC de las máquinas clonadas como se observa en la anterior ilustración, para no tener conflictos con las configuración de red.

Seleccionamos el tipo de clonación *completa*, para realizar una copia exacta de las máquinas:



Posteriormente accedemos a la nueva máquina clonada y modificamos el fichero */etc/hostname* para cambiar el nombre del equipo:

```
root@master:/# nano /etc/hostname

[NUEVO_HOSTNAME]
```

También se debe de alterar el fichero */etc/hosts* y realizar el siguiente cambio:

```
root@[NUEVO_HOSTNAME]:/# nano /etc/hosts

127.0.1.1       [NUEVO_HOSTNAME]
```

Por último vamos a cambiar la IP interna de la máquina clonada, para ello editamos el fichero */etc/network/interfaces*:

```
root@[NUEVO_HOSTNAME]:/# nano /etc/network/interfaces

# Configuración para la red interna
auto eth0
iface eth0 inet static
        address [IP_MAQUINA]
        netmask 255.255.255.0
```

Por último reiniciamos la máquina para que se realicen todos los cambios ejecutados anteriormente.

7.6. Configuración de OpenSSH en master

Posteriormente generamos una clave RSA para permitir el acceso remoto de los servidores *workers*:

```
root@master:/# ssh-keygen

Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa.
Your public key has been saved in /root/.ssh/id_rsa.pub.
The key fingerprint is:
0c:98:6e:f6:1a:32:0e:d4:c9:b6:f8:4a:ea:58:53:e6 root@master
The key's randomart image is:
+---[RSA 2048]----+
|                 |
|     o           |
|    o .          |
|  o..  o         |
| . =*   S        |
|. o*..           |
|.o=.E .          |
|++.+ o           |
|+oo..            |
+-----------------+
```

Copiamos la clave RSA generada anteriormente en los servidores *workers*.

```
root@master:~$ ssh-copy-id -i ~/.ssh/id_rsa.pub root@10.0.0.2
root@master:~$ ssh-copy-id -i ~/.ssh/id_rsa.pub root@10.0.0.3
root@master:~$ ssh-copy-id -i ~/.ssh/id_rsa.pub root@10.0.0.4
```

Adjunto la salida de la última instrucción ejecutada:

```
root@master:/# ssh-copy-id -i ~/.ssh/id_rsa.pub root@10.0.0.4

/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are
already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install
the new keys
root@10.0.0.4's password:

Number of key(s) added: 1

Now try logging into the machine, with:   "ssh 'root@10.0.0.4'"
and check to make sure that only the key(s) you wanted were added.
```

7.7. Configuración de Apache Spark en el servidor master

Vamos a indicar quién es el nodo master y quiénes son los esclavos. Para ello nos dirigimos a la carpeta *conf* y renombramos el fichero denominado slaves.template a slaves. Este fichero contiene las direcciones IP de los nodos esclavos:

```
root@master:/opt/spark/conf# cp slaves.template slaves
root@master:/opt/spark/conf# nano slaves

# A Spark Worker will be started on each of the machines listed below.
# localhost
10.0.0.2
10.0.0.3
10.0.0.4
```

Posteriormente en la carpeta *conf*, renombramos el fichero denominado spark-env.sh.template a spark-env.sh:

```
root@master:/opt/spark/conf# cp spark-env.sh.template spark-env.sh
root@master:/opt/spark/conf# nano spark-env.sh

export SPARK_MASTER_HOST=10.0.0.1 # En mi caso: 10.0.0.1.
export SPARK_WORKER_CORES=1 # Número de cores que se ejecutan en el servidor.
export SPARK_WORKER_INSTANCES=1 # Número de procesos worker por cada nodo.
```

Para comprobamos que Apache Spark funciona correctamente, ejecutamos el siguiente comando para calcular el número Pi con Spark:

```
root@master:/opt/spark/sbin# run-example SparkPi

Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties
17/04/14 19:40:26 INFO SparkContext: Running Spark version 2.1.0
17/04/14 19:40:26 WARN NativeCodeLoader: Unable to load native-hadoop library for your
platform... using builtin-java classes where applicable
17/04/14 19:40:27 WARN Utils: Your hostname, master resolves to a loopback address:
127.0.1.1; using 192.168.1.136 instead (on interface eth0)

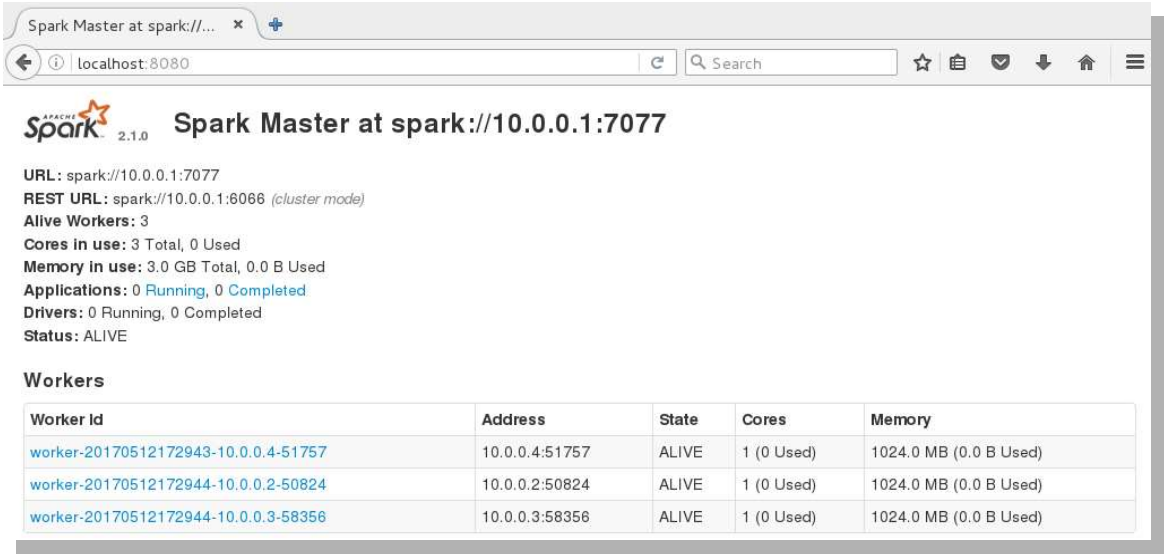
                                    […]
Pi is roughly 3.147915739578698
```

Para comprobar que el clúster funciona correctamente, nos dirigimos a la ubicación */opt/spark/sbin* y ejecutamos el script denominado start-all.sh:

```
root@master:/opt/spark/sbin# start-all.sh

starting org.apache.spark.deploy.master.Master, logging to /opt/spark/logs/spark-root-org.apache.spark.deploy.master.Master-1-master.out
10.0.0.3: starting org.apache.spark.deploy.worker.Worker, logging to /opt/spark/logs/spark-root-org.apache.spark.deploy.worker.Worker-1-worker2.out
10.0.0.2: starting org.apache.spark.deploy.worker.Worker, logging to /opt/spark/logs/spark-root-org.apache.spark.deploy.worker.Worker-1-worker1.out
10.0.0.4: starting org.apache.spark.deploy.worker.Worker, logging to /opt/spark/logs/spark-root-org.apache.spark.deploy.worker.Worker-1-worker3.out
```

Posteriormente accedemos al panel web:



Probamos la ejecución de la función *SparkPi* en el clúster:

```
root@master:/# MASTER=spark://10.0.0.1:7077 run-example SparkPi

Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties
17/04/24 17:26:13 INFO SparkContext: Running Spark version 2.1.0
17/04/24 17:26:14 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
17/04/24 17:26:14 WARN SparkConf:
SPARK_WORKER_INSTANCES was detected (set to '1').
This is deprecated in Spark 1.0+.

Please instead use:
 - ./spark-submit with --num-executors to specify the number of executors
 - Or set SPARK_EXECUTOR_INSTANCES
 - spark.executor.instances to configure the number of instances in the spark config.

17/04/24 17:26:14 WARN Utils: Your hostname, master resolves to a loopback address: 127.0.1.1; using 10.0.0.1 instead (on interface eth0)
17/04/24 17:26:14 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
```

17/04/24 17:26:14 INFO SecurityManager: Changing view acls to: root
17/04/24 17:26:14 INFO SecurityManager: Changing modify acls to: root
17/04/24 17:26:14 INFO SecurityManager: Changing view acls groups to:
17/04/24 17:26:14 INFO SecurityManager: Changing modify acls groups to:
17/04/24 17:26:14 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view permissions: Set(root); groups with view permissions: Set(); users with modify permissions: Set(root); groups with modify permissions: Set()
17/04/24 17:26:14 INFO Utils: Successfully started service 'sparkDriver' on port 59652.
17/04/24 17:26:14 INFO SparkEnv: Registering MapOutputTracker
17/04/24 17:26:14 INFO SparkEnv: Registering BlockManagerMaster
17/04/24 17:26:14 INFO BlockManagerMasterEndpoint: Using org.apache.spark.storage.DefaultTopologyMapper for getting topology information
17/04/24 17:26:14 INFO BlockManagerMasterEndpoint: BlockManagerMasterEndpoint up
17/04/24 17:26:14 INFO DiskBlockManager: Created local directory at /tmp/blockmgr-4f2fbaad-dbeb-4193-b3a0-6a12b0106aa2
17/04/24 17:26:14 INFO MemoryStore: MemoryStore started with capacity 413.9 MB
17/04/24 17:26:15 INFO SparkEnv: Registering OutputCommitCoordinator
17/04/24 17:26:15 INFO Utils: Successfully started service 'SparkUI' on port 4040.
17/04/24 17:26:15 INFO SparkUI: Bound SparkUI to 0.0.0.0, and started at http://10.0.0.1:4040
17/04/24 17:26:15 INFO SparkContext: Added JAR file:/opt/spark/examples/jars/scopt_2.11-3.3.0.jar at spark://10.0.0.1:59652/jars/scopt_2.11-3.3.0.jar with timestamp 1493047575473
17/04/24 17:26:15 INFO SparkContext: Added JAR file:/opt/spark/examples/jars/spark-examples_2.11-2.1.0.jar at spark://10.0.0.1:59652/jars/spark-examples_2.11-2.1.0.jar with timestamp 1493047575474
17/04/24 17:26:15 INFO StandaloneAppClient$ClientEndpoint: Connecting to master spark://10.0.0.1:7077...
17/04/24 17:26:15 INFO TransportClientFactory: Successfully created connection to /10.0.0.1:7077 after 36 ms (0 ms spent in bootstraps)
17/04/24 17:26:15 INFO StandaloneSchedulerBackend: Connected to Spark cluster with app ID app-20170424172615-0000
17/04/24 17:26:15 INFO Utils: Successfully started service 'org.apache.spark.network.netty.NettyBlockTransferService' on port 38322.
17/04/24 17:26:15 INFO NettyBlockTransferService: Server created on 10.0.0.1:38322
17/04/24 17:26:15 INFO BlockManager: Using org.apache.spark.storage.RandomBlockReplicationPolicy for block replication policy
17/04/24 17:26:15 INFO BlockManagerMaster: Registering BlockManager BlockManagerId(driver, 10.0.0.1, 38322, None)
17/04/24 17:26:15 INFO BlockManagerMasterEndpoint: Registering block manager 10.0.0.1:38322 with 413.9 MB RAM, BlockManagerId(driver, 10.0.0.1, 38322, None)
17/04/24 17:26:15 INFO BlockManagerMaster: Registered BlockManager BlockManagerId(driver, 10.0.0.1, 38322, None)
17/04/24 17:26:15 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, 10.0.0.1, 38322, None)
17/04/24 17:26:15 INFO StandaloneAppClient$ClientEndpoint: Executor added: app-20170424172615-0000/0 on worker-20170424164646-10.0.0.3-49533 (10.0.0.3:49533) with 1 cores
17/04/24 17:26:16 INFO StandaloneSchedulerBackend: Granted executor ID app-20170424172615-0000/0 on hostPort 10.0.0.3:49533 with 1 cores, 1024.0 MB RAM
17/04/24 17:26:16 INFO StandaloneAppClient$ClientEndpoint: Executor added: app-20170424172615-0000/1 on worker-20170424164714-10.0.0.2-48513 (10.0.0.2:48513) with 1 cores
17/04/24 17:26:16 INFO StandaloneSchedulerBackend: Granted executor ID app-20170424172615-0000/1 on hostPort 10.0.0.2:48513 with 1 cores, 1024.0 MB RAM
17/04/24 17:26:16 INFO StandaloneAppClient$ClientEndpoint: Executor added: app-20170424172615-0000/2 on worker-20170424164655-10.0.0.4-54490 (10.0.0.4:54490) with 1 cores
17/04/24 17:26:16 INFO StandaloneSchedulerBackend: Granted executor ID app-20170424172615-0000/2 on hostPort 10.0.0.4:54490 with 1 cores, 1024.0 MB RAM
17/04/24 17:26:16 INFO StandaloneAppClient$ClientEndpoint: Executor updated: app-20170424172615-0000/0 is now **RUNNING**
17/04/24 17:26:16 INFO StandaloneAppClient$ClientEndpoint: Executor updated: app-20170424172615-0000/1 is now **RUNNING**
17/04/24 17:26:16 INFO StandaloneAppClient$ClientEndpoint: Executor updated: app-20170424172615-0000/2 is now **RUNNING**
17/04/24 17:26:16 INFO StandaloneSchedulerBackend: SchedulerBackend is ready for scheduling beginning after reached minRegisteredResourcesRatio: 0.0
17/04/24 17:26:16 INFO SharedState: Warehouse path is 'file:/home/usuario/spark-warehouse/'.
17/04/24 17:26:17 INFO SparkContext: Starting job: reduce at SparkPi.scala:38
17/04/24 17:26:17 INFO DAGScheduler: Got job 0 (reduce at SparkPi.scala:38) with 2 output partitions
17/04/24 17:26:17 INFO DAGScheduler: Final stage: ResultStage 0 (reduce at SparkPi.scala:38)
17/04/24 17:26:17 INFO DAGScheduler: Parents of final stage: List()
17/04/24 17:26:17 INFO DAGScheduler: Missing parents: List()
17/04/24 17:26:17 INFO DAGScheduler: Submitting ResultStage 0 (MapPartitionsRDD[1] at map at SparkPi.scala:34), which has no missing parents
17/04/24 17:26:18 INFO MemoryStore: Block broadcast_0 stored as values in memory (estimated size 1832.0 B, free 413.9 MB)
17/04/24 17:26:18 INFO BlockManagerInfo: Added broadcast_0_piece0 in memory on 10.0.0.1:38322 (size: 1172.0 B, free: 413.9 MB)
17/04/24 17:26:18 INFO TaskSchedulerImpl: Adding task set 0.0 with 2 tasks

```
17/04/24 17:26:19 INFO CoarseGrainedSchedulerBackend$DriverEndpoint: Registered executor NettyRpcEndpointRef(null)
(10.0.0.2:58323) with ID 1
17/04/24 17:26:19 INFO TaskSetManager: Starting task 0.0 in stage 0.0 (TID 0, 10.0.0.2, executor 1, partition 0,
PROCESS_LOCAL, 6082 bytes)
17/04/24 17:26:19 INFO CoarseGrainedSchedulerBackend$DriverEndpoint: Registered executor NettyRpcEndpointRef(null)
(10.0.0.4:38009) with ID 2
17/04/24 17:26:19 INFO TaskSetManager: Starting task 1.0 in stage 0.0 (TID 1, 10.0.0.4, executor 2, partition 1,
PROCESS_LOCAL, 6082 bytes)
17/04/24 17:26:19 INFO BlockManagerMasterEndpoint: Registering block manager 10.0.0.2:48312 with 413.9 MB RAM,
BlockManagerId(1, 10.0.0.2, 48312, None)
17/04/24 17:26:19 INFO CoarseGrainedSchedulerBackend$DriverEndpoint: Registered executor NettyRpcEndpointRef(null)
(10.0.0.3:42850) with ID 0
17/04/24 17:26:19 INFO BlockManagerMasterEndpoint: Registering block manager 10.0.0.4:42877 with 413.9 MB RAM,
BlockManagerId(2, 10.0.0.4, 42877, None)
17/04/24 17:26:20 INFO BlockManagerMasterEndpoint: Registering block manager 10.0.0.3:35754 with 413.9 MB RAM,
BlockManagerId(0, 10.0.0.3, 35754, None)
17/04/24 17:26:20 INFO BlockManagerInfo: Added broadcast_0_piece0 in memory on 10.0.0.2:48312 (size: 1172.0 B, free: 413.9
MB)
17/04/24 17:26:20 INFO BlockManagerInfo: Added broadcast_0_piece0 in memory on 10.0.0.4:42877 (size: 1172.0 B, free: 413.9
MB)
17/04/24 17:26:20 INFO TaskSetManager: Finished task 0.0 in stage 0.0 (TID 0) in 1229 ms on 10.0.0.2 (executor 1) (1/2)
17/04/24 17:26:21 INFO DAGScheduler: ResultStage 0 (reduce at SparkPi.scala:38) finished in 2,977 s
17/04/24 17:26:21 INFO TaskSetManager: Finished task 1.0 in stage 0.0 (TID 1) in 1439 ms on 10.0.0.4 (executor 2) (2/2)
17/04/24 17:26:21 INFO TaskSchedulerImpl: Removed TaskSet 0.0, whose tasks have all completed, from pool
17/04/24 17:26:21 INFO DAGScheduler: Job 0 finished: reduce at SparkPi.scala:38, took 3,660659 s
Pi is roughly 3.1399756998784993
                                                    [...]
```

Como se puede observar en la siguiente ilustración se ha completado correctamente la aplicación:

